Chapter 5: Loops

COMP 110
Worked examples, traces, discussion, and quizzes

Learning goals

Trace while, do-while, and for loops
Explain how each iteration makes progress
Use counters, accumulators, and sentinels
Build and trace nested loops
Use break and continue deliberately
Test zero, one, many, and boundary cases

Repeat a task without repeating source lines

```
int count = 0;
while (count < 100) {
    System.out.println("Welcome to Java!");
    count++;
}
```

The body runs 100 times, for count values 0 through 99.

Java: RepeatMessage.java

Three common loop forms

Form	When the condition is tested
while	Before each body execution
do-while	After each body execution
for	Before each body execution, with setup and update in the header

Choose a form that makes the state and stopping rule easy to read.

while: test, run, update, repeat

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
System.out.println("Final count: " + count);
```

A false test leaves the loop and continues after its body.

Java: WhileTrace.java

Trace count < 2

State at test	Action
count = 0	true; print, then count becomes 1
count = 1	true; print, then count becomes 2
count = 2	false; leave the loop

Two body executions require three condition checks.

Java: WhileTrace.java

A while loop may run zero times

```
int count = 5;
while (count < 3) {
    System.out.println("Inside");
    count++;
}
System.out.println("After: " + count);
```

The condition is checked before the first body execution.

Java: ZeroIterations.java

Quiz 1: trace a while loop

```
int i = 1, sum = 0;
while (i <= 3) {
    sum += i;
    i++;
}
```

What are sum and i afterward?
How many times is the condition checked?

Java: WhileQuiz.java

Counter example: print 1 through 10

```
int number = 1;
while (number <= 10) {
    System.out.print(number + " ");
    number++;
}
System.out.println();
```

Check the first included value and the last included value.

Java: PrintOneToTen.java

A different start and endpoint

```
for (int number = 20; number <= 50; number++) {
    System.out.print(number + " ");
}
System.out.println();
```

The inclusive interval 20 through 50 contains 31 integers.

Java: PrintTwentyToFifty.java

The update can skip directly to the next valid value

```
for (int number = 1; number <= 100; number += 2) {
    System.out.print(number + " ");
}
System.out.println();
```

Starting at 1 and adding 2 visits the 50 odd values through 99.

Java: OddNumbers.java

Accumulate a sum one value at a time

```
long sum = 0;
for (int i = 1; i <= n; i++) {
    sum += i;
}
System.out.println("Sum: " + sum);
```

Before iteration i, sum contains 1 through i - 1.

Java: SumOneToN.java

Count input values rather than their magnitudes

```
Scanner input = new Scanner(System.in);
System.out.println("Enter ten integers:");
long sum = 0;
for (int count = 0; count < 10; count++) {
    int value = input.nextInt();
    sum += value;
}
System.out.println("Sum: " + sum);
```

The loop runs ten times even when the entered values are negative or zero.

Java: SumTenNumbers.java

Discussion: how will this loop stop?

Name the state that changes each iteration.
Explain why it moves toward a false condition.

What happens if count++ is missing?
What happens if count-- replaces it
while the condition is count < 100?

Conditional loop: sum the decimal digits

```
int remaining = number;
int sum = 0;
while (remaining > 0) {
    sum += remaining % 10;
    remaining /= 10;
}
System.out.println("Digit sum: " + sum);
```

Remainder extracts a digit; division removes that digit.

Java: DigitSum.java

Build the reverse and add it to the original

```
int remaining = number;
long reverse = 0;
while (remaining > 0) {
    reverse = reverse * 10 + remaining % 10;
    remaining /= 10;
}
long total = number + reverse;
System.out.println(number + " + " + reverse + " = " + total);
```

Multiplying reverse by ten makes room for the next extracted digit.

Java: ReverseAndAdd.java

Trace the reverse of 1024

Digit removed	New reverse / remaining
4	4 / 102
2	42 / 10
0	420 / 1
1	4201 / 0

The loop stops when no unprocessed digits remain.

Java: ReverseAndAdd.java

A sentinel value marks the end of input

```
long sum = 0;
int value = input.nextInt();
while (value != 0) {
    sum += value;
    value = input.nextInt();
}
System.out.println("Sum: " + sum);
```

The zero is tested before accumulation, so it is not processed as data.

Java: SentinelValue.java

Separate the number of values from their sum

```
long sum = 0;
int count = 0;
int score = input.nextInt();
while (score != -1) {
    sum += score;
    count++;
    score = input.nextInt();
}
if (count == 0) System.out.println("No scores");
else System.out.println("Average: " + (double) sum / count);
```

Handle an empty data set before dividing by count.

Java: SentinelAverage.java

An input stream can supply its own stopping rule

```
Scanner input = new Scanner(System.in);
long sum = 0;
while (input.hasNextInt()) {
    sum += input.nextInt();
}
System.out.println("Sum: " + sum);
```

hasNextInt checks for another integer token before nextInt reads it.

Java: EndOfInputSum.java

Quiz 2: counters and sentinels

1. What is the digit sum of 1024?
2. What is the numeric reverse of 120?
3. Should a sentinel contribute to the average?
4. What should an average program do with no values?
5. Which statement makes progress in a digit loop?

Floating-point equality is a fragile stopping rule

```
double item = 1;
int steps = 0;
while (item != 0 && steps < 12) {
    item -= 0.1;
    steps++;
}
System.out.println("Steps: " + steps);
System.out.printf(Locale.US, "Item: %.3f%n", item);
System.out.println("Reached zero: " + (item == 0));
```

The step limit makes this diagnostic example safe to run.

Java: BoundedFloatingLoop.java

Control the count with an integer

```
double sum = 0;
for (int tenths = 10; tenths >= 1; tenths--) {
    sum += tenths / 10.0;
}
System.out.printf(Locale.US, "Sum: %.1f%n", sum);
```

The integer counter fixes exactly ten terms: 1.0 through 0.1.

Java: IntegerControlledSeries.java

do-while tests after the body

```
int count = 5;
do {
    System.out.println("Body: " + count);
    count++;
} while (count < 3);
System.out.println("After: " + count);
```

The body runs once even when the continuation condition will be false.

Java: DoWhileDemo.java

A prompt that must occur at least once

```
int value;
do {
    System.out.println("Enter a value from 1 to 5:");
    value = input.nextInt();
} while (value < 1 || value > 5);
System.out.println("Accepted: " + value);
```

Repeat while the entered integer is outside the allowed range.

Java: InputValidationLoop.java

for puts setup, test, and update together

```
for (int i = 0; i < 2; i++) {
    System.out.println("Welcome to Java!");
}
// i is out of scope here
```

The update runs after a normally completed body, before the next test.

Java: ForTrace.java

Trace for (int i = 0; i < 2; i++)

Stage	State or action
Initialize	i = 0, once
Test at 0; body; update	true; print; i becomes 1
Test at 1; body; update	true; print; i becomes 2
Test at 2	false; leave the loop

Two iterations, two updates, and three tests.

Java: ForTrace.java

Quiz 3: count the body executions

1. for (int i = 1; i <= 5; i++)
2. for (int i = 0; i < 5; i++)
3. for (int i = 0; i <= 5; i++)
4. A while condition is initially false.
5. A do-while condition is false on its first test.

Java: OffByOneQuiz.java

Translate a simple for loop into while

```
int first = 0;
for (int i = 1; i <= 3; i++) first += i;
int second = 0;
int j = 1;
while (j <= 3) {
    second += j;
    j++;
}
System.out.println(first + " " + second);
```

Preserve initialization, the test, and the position of the update.

Java: EquivalentLoops.java

An extra semicolon can detach the intended body

```
int i;
for (i = 0; i < 3; i++); // empty loop body
{
    System.out.println("Printed once: " + i);
}
```

This finishes the empty loop, then prints once with i equal to 3.

Java: ForSemicolonPitfall.java

Legal compact forms can be harder to read

```
for (int i = 1; i <= 3; System.out.println(i++));
for (int i = 0, j = 0; i + j < 6; i++, j++) {
    System.out.println(i + ":" + j);
}
```

Prefer a clear body and a simple update in ordinary course code.

Java: CompactForDemo.java

Discussion: choose the clearest loop form

Print a known number of rows.
Read until a sentinel is entered.
Prompt until a valid numeric range is supplied.

Which loop would you choose for each?
What evidence shows that it will stop?

A nested loop repeats the inner loop for each row

```
for (int row = 1; row <= 2; row++) {
    for (int column = 1; column <= 3; column++) {
        System.out.print(row + ":" + column + " ");
    }
    System.out.println();
}
```

The inner column counter is initialized again for every outer iteration.

Java: NestedLoops.java

Print a 9 by 9 multiplication table

```
for (int row = 1; row <= 9; row++) {
    for (int column = 1; column <= 9; column++) {
        System.out.printf("%4d", row * column);
    }
    System.out.println();
}
```

row chooses the row, column chooses the column, and their product fills a cell.

Java: MultiTable.java

A triangular pattern has a changing inner bound

```
for (int row = 1; row <= 4; row++) {
    for (int star = 1; star <= row; star++) {
        System.out.print("*");
    }
    System.out.println();
}
```

The four rows print 1 + 2 + 3 + 4 = 10 stars.

Java: TrianglePattern.java

Quiz 4: nested loops

1. A 3-row, 4-column rectangle has how many cells?
2. How many newline calls produce its three rows?
3. Why must the inner counter reset?
4. How many stars are in rows 1 through 4
   of the triangle example?

GCD: keep the largest common divisor found

```
int gcd = 1;
for (int candidate = 2; candidate <= first
        && candidate <= second; candidate++) {
    if (first % candidate == 0 && second % candidate == 0) {
        gcd = candidate;
    }
}
System.out.println("GCD: " + gcd);
```

Every common divisor must be no larger than the smaller input.

Java: GreatestCommonDivisor.java

Extension: a shrinking remainder can find the GCD

```
Scanner input = new Scanner(System.in);
System.out.print("Enter two positive ints: ");
int first = input.nextInt();
int second = input.nextInt();
if (first < 1 || second < 1) {
    System.out.println("Positive inputs required");
    return;
}
while (second != 0) {
    int remainder = first % second;
    first = second;
    second = remainder;
}
System.out.println("GCD: " + first);
```

For 24 and 16, the state becomes 16,8 and then 8,0.

Java: EuclidGcd.java

Repeat growth until a threshold is reached

```
double tuition = 10000;
int years = 0;
while (tuition < 20000) {
    tuition *= 1.07;
    years++;
}
System.out.println("Years: " + years);
System.out.printf(Locale.US, "Tuition: %.2f%n", tuition);
```

The first year at or above $20,000 is year 11.

Java: FutureTuition.java

Decimal to hex: process the remainders

```
int remaining = number;
String hex = "";
do {
    int digit = remaining % 16;
    char symbol = digit < 10
        ? (char) ('0' + digit) : (char) ('A' + digit - 10);
    hex = symbol + hex;
    remaining /= 16;
} while (remaining > 0);
System.out.println("Hex: " + hex);
```

The do-while form also produces one digit for input zero.

Java: Dec2Hex.java

Quiz 5: choose the invariant and stopping state

1. What remains true about gcd during its search?
2. Why does the decimal digit loop reach zero?
3. Why does Dec2Hex need to handle zero explicitly?
4. Does tuition doubling happen at year 10 or 11?

break leaves the nearest enclosing loop

```
int number = 0, sum = 0;
while (number < 20) {
    number++;
    sum += number;
    if (sum >= 100) break;
}
System.out.println("Number: " + number);
System.out.println("Sum: " + sum);
```

The addition happens before the threshold check.

Java: TestBreak.java

continue skips the rest of the current body

```
int number = 0, sum = 0;
while (number < 20) {
    number++;
    if (number == 10 || number == 11) continue;
    sum += number;
}
System.out.println("Number: " + number);
System.out.println("Sum: " + sum);
```

The values 10 and 11 are skipped, but the loop keeps running.

Java: TestContinue.java

A basic for still performs its update after continue

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
System.out.println();
```

The body output for 3 is skipped; the update advances to 4.

Java: ForContinue.java

An inner break does not leave the outer loop

```
for (int row = 1; row <= 2; row++) {
    for (int column = 1; column <= 3; column++) {
        if (column == 2) break;
        System.out.println(row + ":" + column);
    }
}
System.out.println("Finished");
```

The inner column loop stops at 2 for each of the two rows.

Java: NestedBreak.java

A one-time guess makes only one decision

```
Scanner input = new Scanner(System.in);
int target = (int) (Math.random() * 101);
System.out.println("Guess 0-100:");
int guess = input.nextInt();
if (guess < target) System.out.println("Too low");
else if (guess > target) System.out.println("Too high");
else System.out.println("Correct: " + target);
```

A loop is needed to ask again after an incorrect guess.

Java: GuessNumberOneTime.java

Repeat guessing until the target is found

```
int attempts = 0;
int guess;
do {
    System.out.println("Guess 0-100:");
    guess = input.nextInt();
    attempts++;
    if (guess < target) System.out.println("Too low");
    else if (guess > target) System.out.println("Too high");
} while (guess != target);
System.out.println("Correct: " + target);
System.out.println("Attempts: " + attempts);
```

The full file generates the target once before entering this loop.

Java: GuessNumber.java

A deliberate indefinite loop can have a clear exit

```
int attempts = 0;
while (true) {
    System.out.println("Guess 0-100:");
    int guess = input.nextInt();
    attempts++;
    if (guess == target) break;
    System.out.println(guess < target ? "Too low" : "Too high");
}
System.out.println("Correct: " + target);
System.out.println("Attempts: " + attempts);
```

while (true) repeats until the successful guess reaches break.

Java: GuessNumberUsingBreak.java

Quiz 6: break, continue, and progress

1. Does break finish the current loop?
2. Does continue skip all future iterations?
3. What happens after continue in a basic for?
4. Does an inner break exit the outer loop?
5. Where should a while progress step go
   if continue might bypass the end of the body?

Check a palindrome from both ends

```
int left = 0, right = text.length() - 1;
boolean palindrome = true;
while (left < right) {
    if (text.charAt(left) != text.charAt(right)) {
        palindrome = false;
        break;
    }
    left++;
    right--;
}
System.out.println("Palindrome: " + palindrome);
```

Stop when the indexes meet, cross, or reveal a mismatch.

Java: Palindrome.java

Trace a palindrome comparison

State for noon	Result
left 0, right 3	n == n; move inward
left 1, right 2	o == o; move inward
left 2, right 1	Stop; all compared pairs matched

A mismatch can stop early because one failed pair is enough.

Java: Palindrome.java

Display the first 50 prime numbers

Try candidate values starting at 2.
Test whether a candidate has a divisor.
Count and print it only if it is prime.
Start a new output line after each ten primes.

The candidate value and prime count are different.

Prime test inside the candidate loop

```
boolean prime = true;
for (int divisor = 2; divisor <= number / divisor; divisor++) {
    if (number % divisor == 0) {
        prime = false;
        break;
    }
}
```

This snippet tests one candidate number that is at least 2.

Java: PrimeNumber.java

Count primes and format the output

```
if (prime) {
    System.out.printf("%4d", number);
    count++;
    if (count % 10 == 0) System.out.println();
}
number++;
```

The outer loop stops on prime count 50, not candidate number 50.

Java: PrimeNumber.java

Practice: prime factors in increasing order

For 120, expect 2, 2, 2, 3, 5.

Try the smallest divisor first.
Divide it out as many times as possible.
Then try the next divisor.
Stop when the remaining factor is prime or 1.

Remove every copy of a discovered factor

```
for (int divisor = 2;
        divisor <= remaining / divisor; divisor++) {
    while (remaining % divisor == 0) {
        System.out.print(divisor + " ");
        remaining /= divisor;
    }
}
if (remaining > 1) System.out.print(remaining);
System.out.println();
```

The remaining value shrinks each time a factor is printed.

Java: PrimeFactors.java

Practice: sum the fraction series

Compute the source exercise:

1/3 + 3/5 + 5/7 + 7/9 + … + 95/97 + 97/99

The numerator starts at 1 and increases by 2.
Each denominator is the numerator plus 2.
Use floating-point division.

A loop generates each fraction

```
double sum = 0;
for (int numerator = 1; numerator <= 97; numerator += 2) {
    sum += (double) numerator / (numerator + 2);
}
System.out.printf(Locale.US, "Sum: %.6f%n", sum);
```

The cast occurs before division so the fraction is retained.

Java: SumSeries.java

Discussion: design a useful loop test set

Test zero, one, and several iterations.
Test a sentinel before any data.
Test a threshold just below and just above the exit.
Test input zero in a digit conversion.
Test a mismatch at the first palindrome pair.

Exit exercise: count and sum digits in text

Read an ASCII line such as Room 204.
Count only the digit characters 0-9.
Add their numeric digit values.
Ignore other characters.

For Room 204, expect count 3 and sum 6.

Exit exercise: one possible solution

```
int count = 0, sum = 0;
for (int i = 0; i < text.length(); i++) {
    char ch = text.charAt(i);
    if (ch >= '0' && ch <= '9') {
        count++;
        sum += ch - '0';
    }
}
System.out.println("Count: " + count);
System.out.println("Sum: " + sum);
```

The bound i < length keeps every charAt access valid.

Java: DigitReport.java

Chapter review

State the initial value and the exit condition.
Show the progress step on every repeating path.
Distinguish tests, iterations, and updates.
Preserve the intended behavior at boundaries.
Use a clear policy for empty input and sentinels.
Trace nested loops and early exits carefully.